

C++ Survival Kit

FRE6883 – Recitation 1

Understand the building blocks before writing a program.

Values $\longrightarrow$ decisions $\longrightarrow$ repetition $\longrightarrow$ functions

Guided explanations here; independent syntax practice in Deck 4.

A program starts with a few familiar pieces

Execution begins in `main()`; statements in its body run in order.

```
#include <iostream>
```

```
int main()
```

```
{
```

```
    double price = 101.25;
```

```
    std::cout << price << std::endl;
```

```
    return 0;
```

```
}
```

```
#include <iostream>
```

Makes the standard input/output declarations available.

`{ ... }` **groups a block.**

Statements such as assignments and function calls end with a semicolon.

```
return 0;
```

Ends `main` and reports successful completion.

Read the program as a sequence

Create a variable, display its value, then finish. Headers are processed before execution.

A type tells C++ what kind of value you mean

Choose a type from the meaning of the quantity.

Type	Example declaration	Use in this course
int	<code>int steps = 4;</code>	Whole-number counts and indices.
double	<code>double price = 101.25;</code>	Prices, rates, and numerical calculations.
bool	<code>bool valid = true;</code>	A yes/no result: true or false.
char	<code>char kind = 'P';</code>	One character, written in single quotes.

The type also affects the operations

An integer cannot store a fractional part. Two integer operands use integer division. A bool naturally expresses the result of a comparison.

A double represents decimal quantities approximately; use it for the course's numerical work.

Declaration, initialization, and assignment

These words describe different events in a variable's life.

```
double cash = 250.0;    // declare and initialize
cash = cash - 12.0;     // assign a new value
const int STEPS = 4;    // initialize; no later reassignment
```

Declaration + initialization

The type and name introduce the variable.
The initializer supplies its first value: cash starts at 250.0.

Assignment

Evaluate the right-hand expression using the current value, then store the result. Cash becomes 238.0.

Initialize local variables before reading them.

A declaration such as `double cash;` alone does not give this local variable a usable initial value.

Arithmetic depends on the operand types

C++ evaluates an expression before assigning its result to a variable.

```
int total = 17;
```

```
int count = 4;
```

```
double truncated = total / count;           // 4.0
```

```
double average = 1.0 * total / count;       // 4.25
```

First division: both operands are int, so the fractional part is discarded.

Second division: multiplication by 1.0 produces a double before division.

Operators	Meaning
-----------	---------

+, -, *, /	Addition, subtraction, multiplication, division.
------------	--

%	Integer remainder: 17 % 4 is 1.
---	---------------------------------

(...)	Group a calculation: price * (1.0 + rate).
---------	--

Multiplication and division take precedence over addition and subtraction.

Input fills a variable; output displays a value

Inside `main()`; include `<iostream>`. Assume the user enters a valid number.

```
double price = 0.0;
std::cout << "Enter a price: ";
std::cin >> price;
std::cout << "Price = " << price << std::endl;
```

`std::cin >> price`

Reads input into the existing variable.
Entering 101.25 replaces the initial 0.0 with 101.25.

`std::cout << price`

Displays the stored value. Chaining `<<` combines text and values; `std::endl` ends the line and flushes output.

Default formatting may print a double such as 100.0 as 100; its type is still double.

A condition selects one block of code

Inside main(). A comparison evaluates to true or false.

```
double orderValue = 750.0;
double fee = 0.0;
if (orderValue >= 500.0)
{
    fee = 1.0;
}
else
{
    fee = 2.0;
}
```

= assigns a value; == compares two values.

Read the condition first.

Here, 750.0 is at least 500.0, so the first block sets fee to 1.0.

Exactly one path runs.

The else block runs only when the condition is false. The braces delimit each block.

Comparisons

<, <=, >, >=, ==, !=

Logical operators combine conditions

Operator	Name	When the result is true
&&	AND	Both conditions are true.
	OR	At least one condition is true.
!	NOT	The condition that follows is false.

Inside main(); price is a double, steps is an int. Include <iostream>.

```
bool valid = (price > 0.0) && (steps > 0);  
if (!valid)  
{  
    std::cout << "Check price and step count";  
}
```

Two equivalent descriptions:

Accept when `price > 0.0 && steps > 0`; reject when `price <= 0.0 || steps <= 0`.

A for loop repeats a block in a precise order

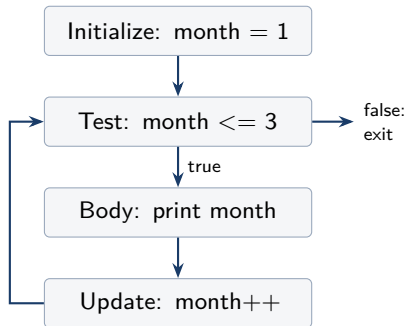
Inside `main()`; include `<iostream>`.

```
for (int month = 1; month <= 3; month++)  
{  
    std::cout << month << " ";  
}
```

Output: 1 2 3

Initialize month once. Test before each execution of the body. Update after the body.

When month becomes 4, the test fails and execution continues after the loop.



Increment and decrement: update, then consider the value

These operators change an integer by one. Prefix/postfix matters when their value is used.

Expression	Change to i	Value supplied by the expression
i++	Add 1	The old value, before the increment.
++i	Add 1	The new value, after the increment.
i--	Subtract 1	The old value, before the decrement.
--i	Subtract 1	The new value, after the decrement.

In the update part of a for loop

The expression's value is unused. Replacing `i++` with `++i` there gives the same traversal. A backward traversal typically uses `i--`.

Example: starting with `i = 4`, `int previous = i++;` leaves `previous = 4` and `i = 5`.

Nested loops express two levels of repetition

One outer iteration runs the entire inner loop. Here n selects a tree row and i selects a node.

```
const int N = 2;
for (int n = 0; n <= N; n++)
{
    for (int i = 0; i <= n; i++)
    {
        // Work on node (n, i).
    }
}
```

Row Nodes visited

0 (0,0)

1 (1,0), (1,1)

2 (2,0), (2,1), (2,2)

The inner counter restarts at zero for each row. Its upper bound depends on the current n .

Match the loop bounds to the structure

Row n has $n + 1$ nodes, numbered 0 through n . These bounds follow the shape of the tree.

An array stores several values under one name

An array contains a fixed number of elements of the same type, stored contiguously.

```
const int SIZE = 4;  
double prices[SIZE] = {98.0, 101.0, 99.0, 104.0};
```

98.0	101.0	99.0	104.0
index 0	index 1	index 2	index 3

Size is a count; an index is a position.

There are four elements. The first is `prices[0]`; the last is `prices[3]`.

The valid range is $0 \leq \text{index} < \text{SIZE}$

Raw array indexing does not check bounds. Accessing `prices[SIZE]` is outside this array and has undefined behavior.

Initialize an array, then traverse its valid indices

Inside `main()`. This fixed-size style follows the course examples.

```
const int SIZE = 4;
double prices[SIZE] = {0.0};
for (int i = 0; i < SIZE; i++)
{
    prices[i] = 100.0 + i;
}
```

After initialization

[0.0, 0.0, 0.0, 0.0]

The first initializer is explicit; the remaining double elements are zero-initialized.

After the loop

[100.0, 101.0, 102.0, 103.0]

Each valid element receives one value. The condition excludes index `SIZE`.

Optional extension: a vector can change its size

Include `<vector>`. `std::vector<double>` denotes a sequence of doubles.

```
// Inside main():  
std::vector<double> prices = {100.0, 102.0};  
prices.push_back(101.0);
```

What changed

`push_back` appends one element: the sequence is now `[100.0, 102.0, 101.0]`.
`prices.size()` reports three elements. Existing elements keep their positions.

Indexing still starts at zero.

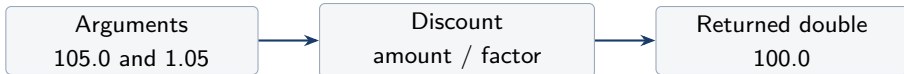
Valid indices are smaller than `size()`; `operator[]` does not automatically check bounds.

Use the course's fixed-size arrays for now. This type also illustrates how a function can receive a larger collection of values.

A function packages a calculation behind a name

A caller supplies arguments; parameters receive them; return supplies the result.

```
double Discount(double amount, double factor)
{
    return amount / factor;
}
// Inside main(); assume factor > 0:
double presentValue = Discount(105.0, 1.05);
```



The return type describes the result; the parameter types describe the inputs.

A return statement ends this function call. Its result can initialize a variable or become part of another expression.

A declaration describes the interface; a definition supplies the body

Declaration: what a caller needs to know

```
double Discount(double amount, double factor);
```

The signature tells the caller which arguments to supply and which result to expect. A declaration ends with a semicolon and has no body.

Definition: how the function does its work

```
double Discount(double amount, double factor)
{
    return amount / factor;
}
```

The braces contain the implementation. Its signature must match the declaration.

Namespaces group names and make their origin explicit

The course uses `fre` for its own functions and `std` for standard-library facilities.

```
namespace fre
{
    double Discount(double amount, double factor);
}
```

Qualified name	How to read it
----------------	----------------

<code>fre::Discount</code>	Discount in the course's <code>fre</code> namespace.
----------------------------	--

<code>std::cout</code>	<code>cout</code> in the standard-library namespace <code>std</code> .
------------------------	--

<code>std::pow</code>	<code>pow</code> in <code>std</code> ; its declaration is available through <code><cmath></code> .
-----------------------	--

`::` selects a name from a scope.

Namespaces allow different groups of code to use the same short names. Use explicit `std::` and `fre::` in these examples.

Headers share declarations; source files define functions

The header is included wherever its declarations are needed, including in its implementation file.

Finance.h

```
#pragma once
namespace fre
{
    double Discount(double amount,
                    double factor);
}

#pragma once prevents repeated inclusion
of this header within one compilation unit.
```

Finance.cpp

```
#include "Finance.h"
namespace fre
{
    double Discount(double amount,
                    double factor)
    {
        return amount / factor;
    }
}
```

Keep the same namespace and signature in both files.

This separation lets several source files call a function through a shared declaration.

Building the program connects calls to definitions

Main.cpp

```
#include "Finance.h"
#include <iostream>
int main()
{
    double pv = fre::Discount(
        105.0, 1.05);
    std::cout << pv << std::endl;
    return 0;
}

g++ Main.cpp Finance.cpp -o pricer
./pricer
```

Build both .cpp files from this folder. Including Finance.h does not compile Finance.cpp.

1. Include the declaration.

Main.cpp can check the argument and result types.

2. Compile the source files.

Each .cpp file is translated separately.

3. Link the program.

The definition in Finance.cpp supplies the function called by Main.cpp.

Pass by value gives a function its own copy

```
double AddFee(double amount)
{
    amount = amount + 2.0;
    return amount;
}

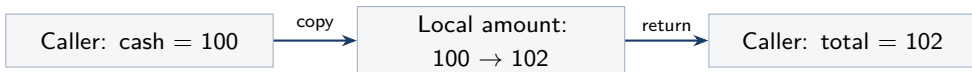
// Inside main():
double cash = 100.0;
double total = AddFee(cash);
```

Two independent variables

The parameter amount starts with a copy of cash's value. Changing amount does not change cash.

After the call

cash = 100.0
total = 102.0



Pass by reference gives another name to the caller's variable

Alternative implementation: the ampersand changes the relationship between parameter and argument.

```
void AddFee(double& amount)
{
    amount = amount + 2.0;
}

// Inside main():
double cash = 100.0;
AddFee(cash);
```

One variable, two names

amount is an alias for cash during this call.
Assigning through amount updates cash itself.

After the call: cash = 102.0.

The return type void means this call produces no returned value.

One shared double: 100.0 → 102.0

Named cash by the caller;
named amount by the function.

A const reference allows reading through an alias

```
bool IsPositive(const double& value)
{
    return value > 0.0;
}
```

Two separate promises in the parameter type

&: value refers to the caller's object instead of holding a separate copy.

const: the function cannot modify that object through value. An assignment to value, or value++, would be rejected.

The caller's own non-const variable can still change later.

Optional collection example: a const reference avoids copying a whole vector while allowing the function to read its elements.

```
double Total(const std::vector<double>& prices);
```

Choose parameter types from the function's job

First decide whether the function must change the caller's object. Then consider copying.

Job	Parameter style	Reason
Read a small number	<code>double amount</code>	A cheap copy; the caller is unaffected.
Update the caller	<code>double& amount</code>	Assignments change the original variable.
Read a larger object	<code>const T& object</code>	Avoid a copy; prevent writes through this parameter.

Here T is shorthand for a type such as `std::vector<double>` (optional extension).

The lecture's `const double` references are valid

For a simple `double` used only as input, passing by value is usually sufficient. A `const` reference matters more when copying a larger object would be costly.

The CRR loop combines familiar building blocks

Price initially holds $N + 1$ terminal payoffs. Assume $N \geq 1$, $R > 0$, and $0 < q < 1$.

```
for (int n = N - 1; n >= 0; n--)  
{  
    for (int i = 0; i <= n; i++)  
    {  
        Price[i] =  
            (q * Price[i + 1]  
             + (1.0 - q) * Price[i]) / R;  
    }  
}
```

Outer loop: move backward in time.

Terminal values at N are known;
compute rows $N - 1$ through 0 .

Inner loop: visit the current row.

Row n contains nodes 0 through n .
Each needs children i and $i + 1$.

Before an assignment: the two array entries hold next-step option values.

After it: $\text{Price}[i]$ holds the current node's value. After all rows, $\text{Price}[0]$ is today's value.

The signed int n can become -1 to end the loop; the largest child index is N , inside the $N + 1$ slots.

Updating in place reuses memory while preserving needed values

One worked row: $n = 1$, $q = 0.5$, $R = 1.0$. Each parent is the average of its two children.

Stage	Slot 0	Slot 1	Slot 2	What happens
Before the loop	2	4	8	All three entries are next-step values.
After $i = 0$	3	4	8	Read 2 and 4, then overwrite slot 0.
After $i = 1$	3	6	8	Read 4 and 8, then overwrite slot 1.

The new active row is [3, 6]

Moving left to right leaves both required children untouched until they are read. Slot 2 remains allocated, but is no longer part of the active row.

Reversing the inner loop would reuse an already updated child. Traversal order is part of the algorithm's correctness.

Count the repeated work to understand complexity

For the backward CRR loop, each node update uses a constant number of arithmetic operations.

Time: count node updates

Rows contain $N, N - 1, \dots, 1$ active nodes:

$$N + (N - 1) + \dots + 1 = \frac{N(N + 1)}{2}.$$

For large N , doubling N roughly quadruples the work: $O(N^2)$ **time**.

Space: count stored values

One array holds $N + 1$ values and is reused for each row.

Doubling N roughly doubles storage: $O(N)$ **space**.

Big-O describes growth as the input size increases.

It is not a timing in seconds. This count covers the backward loop, not all setup or input/output work.

A reading guide for the practice deck

Read a short program in this order, then explain how its values change.

Look at

Establish

Types and initializers	What each variable can store and its starting value.
Expressions and conditions	Operand types, calculation order, and the chosen path.
Loop headers	Start, stopping condition, and update after each iteration.
Array indices	Which elements exist and which values are still needed.
Function parameters	Whether the function receives a copy or an alias.
Return statements	What value the caller receives, if any.
Headers and source files	Where a name is declared and where its body is defined.

Now apply these ideas independently in Deck 4.